

PART-B

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

Selenium is an open-source tool used for automating **web browsers**. It is mainly used to test web applications by performing actions like opening a website, clicking buttons, entering data, and checking results.

Suitable use cases:

1. Testing an e-commerce website, such as login, search, and checkout.
2. Testing a web-based banking application.

Unsuitable use cases:

1. Testing desktop applications such as MS Word or Calculator.
2. Automating mobile apps directly. For mobile app automation, tools like Appium are normally used.

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

Tool	Main purpose
Selenium WebDriver	Used to write and run browser automation scripts using languages such as Python, Java, C#, etc.
Selenium IDE	A browser extension used mainly for recording and playing back browser actions. It is easier for beginners but less flexible than WebDriver.
Selenium Grid	Used to run tests on multiple browsers, machines, or operating systems, often in parallel.

Q3. Explain the roles of the test script, Selenium client library, browser driver, browser, and WebDriver protocol during one command such as driver.get().

The **Selenium client library** provides the Python methods such as driver.get() and converts the Python instruction into a WebDriver request.

The request is sent through the **WebDriver protocol** to the appropriate **browser driver**.

The **browser driver** communicates with the actual **browser** and tells it to navigate to the requested URL.

The browser then opens the page and the result is returned through the driver back to the Selenium client.

So the basic flow is:

Python test script → Selenium client library → WebDriver communication → Browser Driver → Browser

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

A virtual environment creates a separate Python environment for a particular project.

It is recommended because different projects may require different versions of the same package. For example, one project might need an older Selenium version while another needs a newer one.

If packages are installed globally, upgrading or changing a package for one project can affect another project and may cause **dependency conflicts**.

For example, one project could stop working because a globally installed package was upgraded.

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

Selenium Manager is a tool that comes with modern Selenium releases and helps manage the required browser drivers automatically.

Earlier, we commonly had to download the correct ChromeDriver or another browser driver manually and specify its location in the code.

With Selenium Manager, in many normal cases we can simply write:

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

Selenium can detect the browser and manage the appropriate driver automatically.

This makes setup much easier and reduces problems related to manually downloading and maintaining drivers.

Q6. Differentiate `driver.find_element()` and `driver.find_elements()`. What is returned when no element matches?

`find_element()` is used when we want to find **one element**.

```
element = driver.find_element(By.ID, "username")
```

If no matching element is found, it raises a **NoSuchElementException**.

`find_elements()` is used when we want to find **multiple elements**.

```
elements = driver.find_elements(By.TAG_NAME, "input")
```

If there are no matching elements, it returns an **empty list []** instead of throwing `NoSuchElementException`.

Q7. Compare ID, name, class name, CSS selector, and XPath locators. Which locator strategy would you prefer for stable tests, and why?

Different locators identify elements in different ways:

- **ID:** Finds an element using its id. Usually a very good choice if the ID is unique and stable.

- **Name:** Uses the element's name attribute.
- **Class name:** Uses the CSS class of the element, but can be less reliable if classes are dynamically generated or frequently changed.
- **CSS selector:** Provides a flexible way to locate elements using CSS syntax.
- **XPath:** Can locate elements using their structure, attributes, text, and relationships with other elements.

For stable tests, I would generally prefer a **unique and stable ID first**. If a stable ID isn't available, I would usually use a **stable CSS selector**, and XPath when the element requires more complex relationships or conditions.

The important thing is not simply choosing one locator everywhere. The locator should depend on how stable the application's HTML is.

Q8. Differentiate implicit wait, explicit wait, and time.sleep(). Give one appropriate use case for each.

Implicit wait tells Selenium to wait for a certain amount of time when trying to find elements.

```
driver.implicitly_wait(10)
```

It can be useful when elements generally take a short amount of time to appear throughout the application.

Explicit wait waits for a specific condition.

```
WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.ID, "username"))
)
```

This is usually preferred when we know what condition we are waiting for, such as an element becoming visible or clickable.

time.sleep() simply pauses the Python program for a fixed amount of time.

```
time.sleep(3)
```

It can be useful for a quick debugging experiment, but it is generally not preferred in real automation because the application may become ready earlier or later than the fixed time.

Q9. Why should `driver.quit()` normally be placed in a finally block or teardown method? How is it different from `driver.close()`?

`driver.quit()` should be used to properly end the Selenium session and close **all browser windows** opened by that WebDriver session.

Putting it in a finally block or test teardown makes sure it runs even if the test fails due to an exception.

For example:

try:

```
driver.get("https://example.com")
```

finally:

```
driver.quit()
```

`driver.close()` only closes the **currently active browser window/tab**.

So:

- `close()` → closes the current window/tab.
- `quit()` → ends the WebDriver session and closes all associated windows.

Q10. What causes `StaleElementReferenceException`? Describe two robust ways to handle it without hiding a genuine application defect.

This exception usually happens when we found an element earlier, but the page changed and the old element reference is no longer attached to the current DOM.

For example, a page may refresh or JavaScript may replace an element after we initially located it.

Two good ways to handle it are:

1. Locate the element again before using it.

Instead of keeping an old WebElement reference for too long, find the element again after the page changes.

2. Use an explicit wait and locate the element after the expected DOM update.

For example, wait for the new element to become available and then interact with it.

We should not simply catch the exception and ignore it because that could hide a real application problem.

Q11. What is headless browser execution? Identify two benefits and two limitations.

Headless execution means running a browser **without displaying its graphical user interface**.

For example, Chrome can run in headless mode while Selenium still interacts with the website.

Benefits:

1. It can use fewer system resources.
2. It is useful for running automated tests on CI/CD servers where there may be no graphical desktop.

Limitations:

1. Some browser behavior can differ from normal visible execution.
 2. Debugging can be harder because we cannot directly see what the browser is doing.
-

Q12. Explain how browser and driver version mismatch can affect a Selenium session. Is this still relevant when Selenium Manager is used? Justify.

The browser driver needs to be compatible with the browser it controls. If there is an incompatible version, Selenium may fail to create a browser session or may show a **session creation/version compatibility error**.

Selenium Manager has reduced this problem significantly because it can automatically discover the browser and manage the appropriate driver in normal supported setups.

However, version compatibility is **still relevant**. Selenium Manager doesn't mean that version problems can never occur. Issues can still happen because of unusual environments, manually managed drivers, unsupported browser setups, network restrictions, or other configuration problems.

So Selenium Manager makes driver management easier, but it doesn't completely eliminate the concept of browser-driver compatibility.

Q13. Why can a script work on one computer but fail on another? Discuss at least four environment-related reasons.

There can be several reasons:

1. **Different browser versions** — Chrome, Firefox, or Edge may be different versions.
2. **Different Selenium/Python versions** — the script may depend on a particular package version.
3. **Different operating systems** — Windows and Linux can behave differently in some cases.
4. **Different browser drivers or driver configuration** — especially when drivers are manually managed.
5. **Different screen resolution or display settings** — this can affect some UI interactions.
6. **Network or proxy differences** — one computer may have access to a website while another doesn't.

7. **Missing dependencies or environment variables** — required packages or configurations may not be present.

So automation tests should ideally run in a controlled and reproducible environment.

Q14. What information should be collected before reporting a Selenium defect so another person can reproduce it?

I would collect enough information for another tester or developer to reproduce the problem.

Important information includes:

- Test case name and purpose
- Exact steps to reproduce the issue
- Expected result
- Actual result
- URL or application environment
- Browser and browser version
- Operating system
- Python and Selenium versions
- Error message and complete stack trace
- Screenshots or video if useful
- Test data used
- Relevant test code
- Timestamp and test execution details

If possible, I would also mention whether the problem happens **every time or only occasionally**.

Q15. Identify three ethical or security precautions required when automating a website.

Three important precautions are:

1. **Get proper permission before automating a website**, especially if it is not our own application or if the automation could create significant traffic.
2. **Protect sensitive information**, such as usernames, passwords, API keys, and personal data. These should not be hard-coded or exposed in logs.
3. **Avoid excessive requests or aggressive automation**, because it can put unnecessary load on the website or affect other users.